

El gestor de proyectos

M6: Desarrollo de aplicaciones jee con spring framework

|AE1: Utilizar un gestor de proyectos para la administración del ciclo de vida de un proyecto spring

Introducción

En esta lección comenzaremos por comprender qué es exactamente un gestor de proyectos y los beneficios que aporta a los desarrolladores. A continuación, aprenderemos las características generales de Maven y su Modelo de Objeto de Proyecto (POM).

Maven se ha convertido en una herramienta fundamental para el desarrollo de aplicaciones Java y puede mejorar significativamente la eficiencia y organización de nuestros proyectos.

Una parte fundamental de esta lección será el ciclo de vida de compilación de Maven, que nos permitirá comprender las etapas clave del desarrollo de un proyecto: compilación, ejecución de pruebas, instalación del artefacto, limpieza y empaquetamiento. Aprenderemos cómo cada fase del ciclo de vida juega un papel vital en la construcción de nuestra aplicación.

Aprendizaje esperado

Al finalizar esta lección podrás:

- Utilizar un gestor de proyectos para la administración del ciclo de vida de un proyecto Spring.

Qué es un gestor de proyectos y sus beneficios

Un gestor de proyectos es una **aplicación utilizada tanto por empresas como por equipos independientes para gestionar los diferentes elementos y etapas por los que puede pasar un proyecto**. Estos ayudan a declarar y visualizar por cuáles fases ha de pasar un proyecto, qué tareas han de realizarse y quien ha de realizarlas.

Los gestores de proyectos son muy utilizados en el ámbito del desarrollo software permitiendo aplicar de forma más eficiente una metodología y ayudando a mantener el control sobre los diferentes elementos de un proyecto de software.

En general, el uso de un gestor de proyectos de software agiliza y mejora la gestión de proyectos, permitiendo una mayor eficiencia, colaboración, control y seguimiento de las actividades. Ayuda a minimizar los riesgos y a maximizar las posibilidades de éxito en la ejecución de proyectos.

Características generales de Maven

Maven es una herramienta ampliamente utilizada en el desarrollo de aplicaciones Java. Aunque no es un gestor de proyectos en el sentido tradicional, se puede considerar como una herramienta que facilita la gestión de proyectos de software.

Maven se utiliza principalmente para la **construcción, gestión de dependencias y generación de informes en proyectos Java**. Proporciona una estructura y una convención para organizar el código fuente, las bibliotecas de terceros y los recursos del proyecto.

Algunas de las funcionalidades de Maven que están relacionadas con la gestión de proyectos son las siguientes:

1. **Gestión de dependencias:** Maven permite especificar las dependencias de tu proyecto en un archivo de configuración llamado "**pom.xml**". Maven descarga automáticamente las bibliotecas y las incorpora al proyecto, lo que facilita la gestión de las dependencias externas. También resuelve conflictos de versiones y garantiza la coherencia entre las dependencias.

2. **Construcción y compilación:** Maven se encarga de la construcción del proyecto, incluyendo la compilación del código fuente, la ejecución de pruebas unitarias y la generación de artefactos, como archivos JAR o WAR. Con un solo comando, Maven gestiona la construcción completa del proyecto y sus componentes, lo que facilita la integración y la automatización del proceso.
3. **Estructura del proyecto:** Maven sigue una estructura de directorios estándar para organizar el código fuente, los recursos y las pruebas del proyecto. Esto proporciona una convención que facilita la navegación y la comprensión del proyecto para los desarrolladores y los miembros del equipo. Además, permite una fácil integración con herramientas de desarrollo y control de versiones.
4. **Generación de informes:** Maven incluye la capacidad de generar informes detallados sobre el proyecto, como el informe de cobertura de pruebas, el informe de análisis estático del código o el informe de dependencias. Estos informes proporcionan información valiosa para la toma de decisiones y la evaluación de la calidad del proyecto.

Modelo de Objeto de Proyecto (POM)

Un modelo de objeto de proyecto o **POM** es la **unidad de trabajo fundamental en Maven**. Es un archivo XML que contiene información sobre el proyecto y los detalles de configuración utilizados por Maven para construir el proyecto. Contiene valores predeterminados para la mayoría de los proyectos.

Ejemplos de esto es el directorio de compilación, que es **target**; el directorio fuente, que es **src/main/java**; el directorio fuente de prueba, que es **src/test/java**; etcétera. Al ejecutar una tarea u objetivo, Maven busca el POM en el directorio actual. Lee el POM, obtiene la información de configuración necesaria y luego ejecuta el objetivo.

Algunas de las configuraciones que se pueden especificar en el POM son las dependencias del proyecto, los complementos o los objetivos que se pueden ejecutar, los perfiles de compilación, etc. También se puede especificar otra información, como la versión del proyecto, la descripción, los desarrolladores, las listas de correo y demás.

Los requisitos mínimos para un POM son los siguientes:

- **project** raíz
- **modelVersion**- debe establecerse en 4.0.0
- **groupId**- el id del grupo del proyecto.
- **artifactId**- la identificación del artefacto (proyecto)
- **version**- la versión del artefacto bajo el grupo especificado

Creación de un proyecto Java: Spring Framework

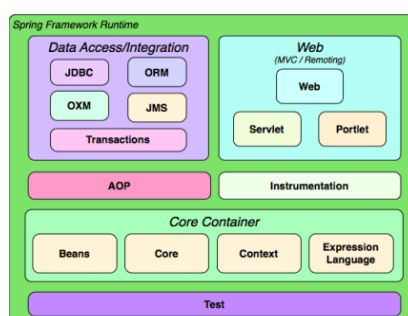
Spring es un framework alternativo al stack de tecnologías estándar en aplicaciones JavaEE. Popularizó ideas como la inyección de dependencias o el uso de objetos convencionales (POJOs) como objetos de negocio.

Spring es el framework más **popular para el desarrollo de aplicaciones empresariales en Java**, para crear código de alto rendimiento, liviano y reusable. Su finalidad es **estandarizar, agilizar, manejar y resolver** los problemas que puedan ir surgiendo en el trayecto de la programación.

Spring, ofrece como elemento clave la **inyección de dependencias** a nuestro proyecto, pero existen otras funcionalidades también muy útiles:

- **Core container:** proporciona inyección de dependencias e inversión de control.
- **Web:** nos permite crear controladores Web, tanto de vistas MVC como aplicaciones REST. Esto facilita en gran medida la programación basada en MVC (Modelo Vista Controlador)
- **Acceso a datos:** abstracciones sobre JDBC, ORMs como Hibernate, sistemas OXM (Object XML Mappers), JSM y transacciones.
- **Instrumentación:** proporciona soporte para la instrumentación de clases.
- **Pruebas de código:** contiene un framework de testing, con soporte para JUnit y TestNG y todo lo necesario para probar los mecanismos de Spring.

Estos módulos son opcionales, por lo que podemos utilizar los que necesitemos



Spring MVC

Es un subproyecto Spring que está dirigido a facilitar y optimizar el proceso creación de aplicaciones web utilizando el patrón **Modelo Vista Controlador**.

MVC es un patrón de diseño que se estructura mediante tres componentes: **modelo, vista y controlador**. Este patrón tiene como principio la separación de cada uno de los componentes en diferentes objetos, esto significa que los componentes no se pueden combinar dentro de una misma clase. Sirve para clasificar la información, la lógica del sistema y la interfaz que se le presenta al usuario.

1. **Modelo:** Esta capa representa todo lo que tiene que ver con el acceso a datos: guardar, actualizar, obtener datos, etc. Básicamente son las clases Java y parte de la lógica de negocio. No contiene ninguna lógica que describa cómo se deben presentar los datos a un usuario.
2. **Vista:** Este componente presenta los datos del modelo al usuario. La vista sabe cómo acceder a los datos del modelo, pero no sabe que significa esta información o que puede hacer el usuario para manipularla.
3. **Controlador:** Este componente se encarga de gestionar, atender y procesar las instrucciones que se reciben. **El controlador es el encargado de conectar el modelo con las vistas.**

El controlador recibe eventos generados por el usuario desde las vistas y se encarga de direccionar la petición al modelo, recibir los resultados y entregarlos a la vista para que pueda mostrarlos.

Spring Boot

Es una herramienta que nace con la finalidad de simplificar aún más el desarrollo de aplicaciones basadas en el framework Spring. Spring Boot busca que el desarrollador se centre únicamente en el desarrollo de la solución, olvidándose por completo de la compleja configuración que actualmente tiene Spring Core para poder funcionar.

Manejo de dependencias con Maven

Maven es un gestor de dependencias. En nuestro proyecto podemos indicar que necesitamos un **jar** (por ejemplo, *log4j* o el conector de MySQL) y Maven es capaz de buscar esos jar en internet y descargarlos automáticamente. Incluso si alguno de esos jar necesita de otros jar para funcionar, Maven "tira del hilo" y va descargando todos los jar que sean necesarios.

Las dependencias se recopilan en el archivo **pom.xml**, dentro de una etiqueta **<dependencies>**.

Un repositorio en Maven contiene artefactos de compilación y dependencias de diferentes tipos.

Repositorios Maven: locales y remotos

El repositorio **local** es un directorio en la computadora donde se ejecuta Maven. Almacena en caché las descargas remotas y contiene artefactos de compilación temporales que aún no ha publicado.

Los repositorios **remotos** se refieren a cualquier otro tipo de repositorio, al que se accede mediante una variedad de protocolos como `file://` y `https://`.

Estos repositorios pueden ser un repositorio verdaderamente remoto configurado por un tercero para proporcionar sus artefactos para descargar (por ejemplo, `repo.maven.apache.org`). Otros repositorios "remotos" pueden ser repositorios internos configurados en un servidor de archivos o `HTTP` dentro de su empresa, utilizados para compartir artefactos privados entre equipos de desarrollo y para lanzamientos.

Los repositorios locales y remotos están estructurados de la misma manera para que los scripts se puedan ejecutar en cualquier lado o se puedan sincronizar para su uso sin conexión. Sin embargo, el diseño de los repositorios es completamente transparente para el usuario de Maven.

Maven: Ciclo de vida de compilación.

Maven se basa en el concepto central de un ciclo de vida de construcción. Esto significa que el proceso para construir y distribuir un artefacto en particular (proyecto) está claramente definido.

Para la persona que construye un proyecto, esto significa que solo es necesario aprender un pequeño conjunto de comandos para construir cualquier proyecto de Maven, y el POM se asegurará de que obtenga los resultados que desea.

Hay tres ciclos de vida de compilación integrados: *default*, *clean* y *site*. El ciclo de vida **default** maneja la **implementación** de su proyecto, el ciclo de vida **clean** maneja la **limpieza** del proyecto, mientras que el ciclo de vida **site** maneja la **creación** del sitio web de su proyecto.

Cada uno de estos ciclos de vida de construcción está definido por una lista diferente de fases de construcción, donde una fase de construcción representa una etapa en el ciclo de vida.

Por ejemplo, el ciclo de vida **predeterminado (default)** consta de las siguientes fases (para obtener una lista completa de las fases del ciclo de vida, consulta la [Referencia del ciclo de vida](#) en la documentación oficial de Apache Maven):

- **validate**- validar que el proyecto sea correcto y que toda la información necesaria esté disponible.
- **compile**- compilar el código fuente del proyecto.
- **test**- probar el código fuente compilado utilizando un marco de prueba de unidad adecuado. Estas pruebas no deberían requerir que el código sea empaquetado o implementado.
- **package**- tomar el código compilado y empaquetarlo en su formato distribuible, como un JAR.
- **verify**- ejecutar cualquier verificación de los resultados de las pruebas de integración para garantizar que se cumplan los criterios de calidad.
- **install**- instalar el paquete en el repositorio local, para usarlo como dependencia en otros proyectos localmente
- **deploy**- Realizado en el entorno de compilación, copia el paquete final en el repositorio remoto para compartirlo con otros desarrolladores y proyectos.

Estas fases del ciclo de vida (más las otras fases del ciclo de vida que no se muestran aquí) se ejecutan secuencialmente para completar el ciclo de vida **default**. Dadas las fases del ciclo de vida anteriores, esto significa que cuando se usa el ciclo de vida default, Maven primero validará el proyecto, luego intentará compilar las fuentes, ejecutarlas contra las pruebas, empaquetar los binarios (por ejemplo, jar), ejecutará pruebas de integración contra esos paquete, verificará las pruebas de integración, instalará el paquete verificado en el repositorio local y luego implementará el paquete instalado en un repositorio remoto.

Cada uno de estos ciclos de vida de construcción está definido por una lista diferente de fases (phases), donde cada fase representa un estado en el ciclo de vida.

Clean Lifecycle	Default Lifecycle		Site Lifecycle
pre-clean	validate	test-compile	pre-site
clean	initialize	process-test-classes	site
post-clean	generate-sources	test	post-site
	process-sources	prepare-package	site-deploy
	generate-resources	package	
	process-resources	pre-integration-test	
	compile	integration-test	
	process-classes	post-integration-test	
	generate-test-sources	verify	
	process-test-sources	install	
	generate-test-resources	deploy	
	process-test-resources		

Cada una de estas fases se ejecutan secuencialmente para completar el ciclo de vida.

La ejecución de los comandos Maven se invocan por consola utilizando el prefijo **mvn** seguido de la fase que deseamos ejecutar:

mvn package

Este comando hará que se ejecuten **las fases previas** a package, en el orden establecido en el Ciclo de Vida (en este caso primero la fase compile y luego la fase test), y finalmente se ejecutará la fase package. Por lo tanto, se debe tener en cuenta que al ejecutar una fase, Maven previamente ejecutará las fases precedentes.

Cierre

A lo largo de esta lección, hemos visto el concepto, las características y los beneficios del gestor de proyectos Maven, así como su Modelo de Objeto de Proyecto (POM). También tuvimos una introducción a los conceptos de Spring MVC y Spring Boot.

Además, aprendiste sobre el ciclo de vida de compilación de Maven, comprendiendo las etapas clave para la construcción exitosa de aplicaciones.

Recuerda que Maven es una herramienta poderosa que puede ahorrarte tiempo y esfuerzo en tus proyectos de desarrollo. Con su capacidad para gestionar dependencias y facilitar la construcción de proyectos, te convierte en un desarrollador más eficiente y organizado.

¡Nos vemos de nuevo, más adelante! 🙌

Referencias

- Apache Software Foundation. (n.d.). Introduction to the POM. Maven. <https://maven.apache.org/guides/introduction/introduction-to-the-pom.html>
- Apache Software Foundation. (n.d.). Introduction to the build lifecycle. Maven. <https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html>
- Apache Maven (s.f.). Ciclo de vida de Maven. Jcodepoint. Retrieved from <https://jcodepoint.com/maven/ciclo-de-vida-de-maven/#:~:text=El%20ciclo%20de%20vida...>
- Arquitectura Java. (s.f.). ¿Qué es Spring Framework? Arquitectura Java. Recuperado de <https://www.arquitecturajava.com/que-es-spring-framework/>
- Chuidiang. (s.f.). Dependencias con Maven. ChuWiki. Recuperado de https://chuidiang.org/index.php?title=Dependencias_con_maven

¡Muchas gracias!

Nos vemos en la próxima lección

